

02_data_exploration

May 31, 2026

```
[ ]: import pandas as pd
import numpy as np

df = pd.read_csv('data/clean/db1b_agg.csv')
print(f"Loaded: {df.shape[0]:,} rows")
df.head()
```

Failed to start the Kernel.

Unable to start Kernel 'Python 3.12 (spirit) (Python 3.12.12)' due to a timeout,
↪waiting for the ports to get used.

View Jupyter [<log for further details.](command:jupyter.viewOutput)

```
[ ]: import numpy as np
import pandas as pd

def cluster_robust_vcov(X, resid, cluster_ids, n_absorbed=0):
    """
    Cluster-robust (sandwich) covariance for OLS or within-FE OLS.
    X          : (N,k) regressors actually used in the final OLS. For FE models
                  these are the WITHIN-transformed regressors (no FE dummies).
    resid      : (N,) residuals from that OLS.
    cluster_ids: (N,) cluster label per row (here: route).
    n_absorbed: number of FE parameters partialled out (for the dof correction).
    """
    X = np.asarray(X, float); resid = np.asarray(resid, float)
    N, k = X.shape
    bread = np.linalg.inv(X.T @ X)
    clusters = np.unique(cluster_ids)
    meat = np.zeros((k, k))
    for g in clusters:
        m = cluster_ids == g
        s = X[m].T @ resid[m]          # (k,)
        meat += np.outer(s, s)
    G = len(clusters)
```

```

    k_total = k + n_absorbed
    c = (G / (G - 1)) * ((N - 1) / (N - k_total))    # Stata-style small-sample
↪adj.
    return c * (bread @ meat @ bread)

def demean(frame, value_cols, fe_cols, n_iter=100, tol=1e-12):
    """Iteratively demean value_cols by each FE dimension (within transform)."""
    work = frame[value_cols + fe_cols].copy().reset_index(drop=True)
    for _ in range(n_iter):
        maxch = 0.0
        for fe in fe_cols:
            for c in value_cols:
                gm = work.groupby(fe)[c].transform('mean')
                new = work[c] - gm
                maxch = max(maxch, float((new - work[c]).abs().max()))
                work[c] = new
        if maxch < tol:
            break
    return work[value_cols]

def run_fe(frame, value_cols, fe_cols, cluster_col='RouteID'):
    """OLS on within-demeaned data with cluster-robust SEs.
       value_cols[0] = dependent var; value_cols[1] = the DiD interaction."""
    dm = demean(frame, value_cols, fe_cols)
    y = dm[value_cols[0]].values
    X = dm[value_cols[1:]].values
    beta = np.linalg.lstsq(X, y, rcond=None)[0]
    resid = y - X @ beta
    n_abs = sum(frame[fe].nunique() for fe in fe_cols)
    V = cluster_robust_vcov(X, resid, frame[cluster_col].values,
↪n_absorbed=n_abs)
    se = np.sqrt(np.diag(V)); t = beta / se
    return dict(names=value_cols[1:], beta=beta, se=se, t=t, n=len(y),
                n_routes=frame['RouteID'].nunique())

```

```

[ ]: NK = 'NK'

# Routes Spirit served in 2022 vs 2025
r2022 = df[(df.TkCarrier==NK)&(df.Year==2022)][['Origin', 'Dest']].
↪drop_duplicates()
r2025 = df[(df.TkCarrier==NK)&(df.Year==2025)][['Origin', 'Dest']].
↪drop_duplicates()

# 176 exited routes (in 2022, not in 2025)
exited = r2022.merge(r2025, on=['Origin', 'Dest'], how='left', indicator=True)
exited = exited[exited._merge=='left_only'][['Origin', 'Dest']]
exited_set = set(map(tuple, exited.values))

```

```

# 86 "strong-exit" routes: Spirit avg >=500 pax/qtr in 2022, and exited
strong = (df[(df.TkCarrier==NK)&(df.Year==2022)]
          .groupby(['Origin', 'Dest']).Passengers.mean().reset_index())
strong = strong[strong.Passengers>=500][['Origin', 'Dest']]
strong = strong.merge(r2025, on=['Origin', 'Dest'], how='left', indicator=True)
strong = strong[strong._merge=='left_only'][['Origin', 'Dest']]
strong_set = set(map(tuple, strong.values))

ever_set = set(map(tuple, r2022.values))    # any Spirit presence in 2022

# Collapse non-Spirit carriers to the route-quarter level (passenger-weighted
↳fare)
o = df[df.TkCarrier!=NK].copy()
o['FareXPax'] = o.AvgFare * o.Passengers
g = (o.groupby(['Year', 'Quarter', 'Origin', 'Dest'])
     .agg(FareXPax=('FareXPax', 'sum'), TotalPax=('Passengers', 'sum'),
          NumCarriers=('TkCarrier', 'nunique'),
          ↳Distance=('NonStopMiles', 'first'))
     .reset_index())
g['WtdFare'] = g.FareXPax / g.TotalPax
g['LogFare'] = np.log(g.WtdFare)
g['LogPax'] = np.log(g.TotalPax + 1)
g['LogDist'] = np.log(g.Distance)
g['RouteID'] = g.Origin + '_' + g.Dest
g['rt'] = list(map(tuple, g[['Origin', 'Dest']].values))
g['SpiritExited'] = g.rt.isin(exited_set).astype(int)    # 176-route treatment
g['StrongExit'] = g.rt.isin(strong_set).astype(int)    # 86-route treatment
g['SpiritEver'] = g.rt.isin(ever_set).astype(int)
g['Control'] = (g.SpiritEver == 0).astype(int)
g['Post'] = ((g.Year>=2025)|((g.Year==2024)&(g.Quarter==4))).astype(int)
g['YQ_num'] = (g.Year-2022)*4 + g.Quarter
g['YQ'] = g.Year.astype(str)+'Q'+g.Quarter.astype(str)
g['Exit176_x_Post'] = g.SpiritExited * g.Post
g['Strong_x_Post'] = g.StrongExit * g.Post

# Matched control: never-Spirit routes with >=2 carriers and distance 400-2000
↳mi
rs = (g.groupby('RouteID').agg(avg_carr=('NumCarriers', 'mean'),
                              dist=('Distance', 'first')).reset_index())
matched_routes = set(rs[(rs.avg_carr>=2)&(rs.dist>=400)&(rs.dist<=2000)].
↳RouteID)
g['MatchedControl'] = ((g.Control==1)&(g.RouteID.isin(matched_routes))).
↳astype(int)

df_reg = g
print(f"Route-quarter observations: {len(df_reg):,}")

```

```

print(f"Routes: {df_reg.RouteID.nunique():,} | Quarters: {df_reg.YQ.nunique():,}")
print(f"Exited (176) flagged route-qtrs: {df_reg.SpiritExited.sum():,}")
print(f"Strong-exit (86) flagged route-qtrs: {df_reg.StrongExit.sum():,}")
print(f"Strong-exit routes: {len(strong_set)} | Exited routes:
↳ {len(exited_set)}")
print(f"Matched-control route-qtrs: {df_reg.MatchedControl.sum():,}")

```

```

[ ]: m1 = df_reg
y = m1.LogFare.values
X = np.column_stack([np.ones(len(m1)), m1.SpiritExited, m1.Post,
                    m1.Exit176_x_Post, m1.LogDist, m1.LogPax])
names = ['Intercept', 'SpiritExited', 'Post', 'SpiritExited *
↳ Post', 'LogDist', 'LogPax']
beta = np.linalg.lstsq(X, y, rcond=None)[0]
resid = y - X @ beta
V = cluster_robust_vcov(X, resid, m1.RouteID.values, n_absorbed=0)
se = np.sqrt(np.diag(V)); t = beta / se

print("MODEL 1: OLS, 176-route treatment, cluster-robust SE (by route)")
print(f"{'Variable':<22}{'Coef':>10}{'ClustSE':>10}{'t':>8}")
for i,nm in enumerate(names):
    print(f"{'nm':<22}{'beta[i]:>10.4f}{'se[i]:>10.4f}{'t[i]:>8.2f}")
b, s = beta[3], se[3]
print(f"\nDiD coef {b:+.4f} -> fare effect {(np.exp(b)-1)*100:+.1f}% "
      f"| 95% CI [{(np.exp(b-1.96*s)-1)*100:+.1f}%, {(np.exp(b+1.96*s)-1)*100:+.
↳ 1f}%]")
m1_res = (b, s)

```

```

[ ]: # M2: route FE only, 86-route treatment (Post stays in; not absorbed)
M2 = run_fe(df_reg, ['LogFare', 'Strong_x_Post', 'Post', 'LogPax'], ['RouteID'])

# M3 (= paper's "Model 2"): route + quarter FE, 86-route treatment
M3 = run_fe(df_reg, ['LogFare', 'Strong_x_Post', 'LogPax'], ['RouteID', 'YQ'])

# M4: route + quarter FE, 86-route treatment, MATCHED control only
m4_sample = df_reg[(df_reg.StrongExit==1) | (df_reg.MatchedControl==1)].copy()
M4 = run_fe(m4_sample, ['LogFare', 'Strong_x_Post', 'LogPax'], ['RouteID', 'YQ'])

def line(tag, res):
    b, s = res['beta'][0], res['se'][0]
    pct = (np.exp(b)-1)*100
    lo = (np.exp(b-1.96*s)-1)*100
    hi = (np.exp(b+1.96*s)-1)*100
    print(f"{'tag':<34}{'b':>9.4f}{'s':>9.4f}{'res['t'][0]:>7.2f}{'pct':>8.1f}%"
          f"    [{lo:+.1f}%, {hi:+.1f}%]    N={res['n']:,}")

b1,s1 = m1_res

```

```

pct1=(np.exp(b1)-1)*100; lo1=(np.exp(b1-1.96*s1)-1)*100; hi1=(np.exp(b1+1.
↳96*s1)-1)*100
print("Effect of Spirit exit on competitor fares (cluster-robust SE by route)")
print(f"{'Model':<34}{'Coef':>9}{'SE':>9}{'t':>7}{'%Δ':>8}    95% CI")
print("-"*88)
print(f"{'M1: OLS, 176 routes, no FE':<34}{'b1':>9.4f}{'s1':>9.4f}{'b1/s1':>7.2f}"
      f"{'pct1':>8.1f}%    [{lo1:+.1f}%, {hi1:+.1f}%]")
line('M2: Route FE, 86 routes', M2)
line('M3: Route+Qtr FE, 86 routes', M3)
line('M4: Route+Qtr FE, 86, matched ctrl', M4)

```

```

[ ]: import matplotlib.pyplot as plt

REF = 11  # YQ_num for 2024Q3, the quarter just BEFORE the 2024Q4 filing
es = df_reg.copy()
es_cols, es_map = [], {}
for q in sorted(es.YQ_num.unique()):
    if q == REF:
        continue
    col = f'ES_{q}'
    es[col] = es.StrongExit * (es.YQ_num == q).astype(int)
    es_cols.append(col); es_map[col] = q

ES = run_fe(es, ['LogFare'] + es_cols + ['LogPax'], ['RouteID', 'YQ'])
coef = dict(zip(ES['names'], ES['beta']))
sek = dict(zip(ES['names'], ES['se']))

# assemble for plotting (reference quarter pinned at 0)
xs = sorted(es.YQ_num.unique())
pts, los, his = [], [], []
for q in xs:
    if q == REF:
        pts.append(0); los.append(0); his.append(0)
    else:
        col = f'ES_{q}'
        b, s = coef[col], sek[col]
        pts.append(b); los.append(b-1.96*s); his.append(b+1.96*s)

labels = (es[['YQ_num', 'YQ']].drop_duplicates().sort_values('YQ_num').YQ.
↳tolist())
fig, ax = plt.subplots(figsize=(11,5))
ax.errorbar(xs, pts, yerr=[np.array(pts)-np.array(los), np.array(his)-np.
↳array(pts)],
            fmt='o', color='#457b9d', capsize=4, linewidth=1.5)
ax.axhline(0, color='black', lw=0.8)
ax.axvline(11.5, color='red', ls='--', alpha=0.6)
ax.text(11.6, max(his)*0.9, 'Ch.11 filing', color='red', fontsize=9)

```

```

ax.set_xticks(xs); ax.set_xticklabels(labels, rotation=45, ha='right',
    ↪ fontsize=8)
ax.set_ylabel('StrongExit × Quarter coefficient (log fare)')
ax.set_title('Event Study: Fares on Spirit-Exited vs. Control Routes\n'
    '(2024Q3 = reference; flat pre-period supports parallel trends)')
ax.grid(axis='y', alpha=0.3)
plt.tight_layout(); plt.savefig('figure_event_study.png', dpi=150); plt.show()

print("Pre-period (should be ~0 and individually insignificant if PT holds):")
for q in xs:
    if q < REF and q != REF:
        col=f'ES_{q}'; b,s=coef[col],sek[col]
        flag = ' ' if abs(b/s)<1.96 else ' <-- significant pre-trend!'
        print(f" {labels[xs.index(q)]}: {b:+.4f} (t={b/s:+.2f}){flag}")

```

```

[ ]: d = df[df.TkCarrier!='NK'].copy()
d['rt'] = list(map(tuple, d[['Origin','Dest']].values))
d['Period'] = np.where((d.Year>=2025)|((d.Year==2024)&(d.
    ↪ Quarter==4)), 'Post', 'Pre')
d['grp'] = np.where(d.rt.isin(exited_set), 'Exited',
    np.where(d.rt.isin(ever_set), 'Stayed', 'Control'))

def pre_post(carrier, grp):
    sub = d[(d.TkCarrier==carrier)&(d.grp==grp)]
    pre = sub[sub.Period=='Pre'].Passengers.mean()
    post = sub[sub.Period=='Post'].Passengers.mean()
    return pre, post

print("Carrier growth on EXITED routes vs. the same carrier on CONTROL routes")
print(f"{'Carr':<6}{'ExitedΔ%':>10}{'ControlΔ%':>11}{'DiD (pp)':>10}")
print("-"*40)
for c in ['DL', 'AA', 'WN', 'F9', 'B6']:
    ep, eo = pre_post(c, 'Exited'); cp, co = pre_post(c, 'Control')
    e_pct = (eo/ep-1)*100 if ep else np.nan
    c_pct = (co/cp-1)*100 if cp else np.nan
    print(f"{'c':<6}{'e_pct':>9.1f}%{'c_pct':>10.1f}%{'e_pct-c_pct':>9.1f}")

# Extensive-margin check you already had: non-Spirit carrier COUNT DiD
cc = (d.groupby(['Year', 'Quarter', 'rt', 'grp']).TkCarrier.nunique()
    .reset_index().rename(columns={'TkCarrier': 'NumCarr'}))
cc['Period'] = np.where((cc.Year>=2025)|((cc.Year==2024)&(cc.
    ↪ Quarter==4)), 'Post', 'Pre')
tab = cc[cc.grp.isin(['Exited', 'Control'])].groupby(['grp', 'Period']).NumCarr.
    ↪ mean().unstack()
did = (tab.loc['Exited', 'Post']-tab.loc['Exited', 'Pre']) - \
    (tab.loc['Control', 'Post']-tab.loc['Control', 'Pre'])
print(f"\nNon-Spirit carrier-count DiD: {did:+.3f} "

```

```

        f"(near 0 => little NEW entry; backfill was intensive-margin)")
print(tab.round(2).to_string())

```

```

[ ]: results = {
    'M1: OLS\n(176, no FE)':          m1_res,
    'M2: Route FE\n(86)':            (M2['beta'][0], M2['se'][0]),
    'M3: Route+Qtr FE\n(86)':        (M3['beta'][0], M3['se'][0]),
    'M4: Route+Qtr FE\n(86, matched)': (M4['beta'][0], M4['se'][0]),
}
labels = list(results.keys())
coefs  = [v[0] for v in results.values()]
ses    = [v[1] for v in results.values()]

fig, ax = plt.subplots(figsize=(9,4))
yp = range(len(labels))
ax.errorbar(coefs, yp, xerr=[1.96*s for s in ses], fmt='o', color='#457b9d',
            markersize=8, capsize=6, linewidth=2)
ax.axvline(0, color='black', lw=0.8)
ax.set_yticks(list(yp)); ax.set_yticklabels(labels, fontsize=9)
ax.set_xlabel('Coef on Treatment × Post (log fare, 95% CI, clustered by route)')
ax.set_title('Effect of Spirit Exit on Competitor Fares Across Specifications')
for i,(c,s) in enumerate(zip(coefs,ses)):
    ax.text(c+1.96*s+0.002, i, f'{{(np.exp(c)-1)*100:+.1f}}%', va='center',
           ↪ fontsize=9)
ax.invert_yaxis(); ax.grid(axis='x', alpha=0.3)
plt.tight_layout(); plt.savefig('figure3_coefficients.png', dpi=150); plt.show()

```